

The Second Half: 语言 Agent、奖励机制与智能边界

Zhang Xiaojun 对话 OpenAI 研究员姚顺雨

基于 Zhang Xiaojun Podcast 公开视频、GPU 转写稿、章节结构与自制概念图整理

2025-09-11



视频标题: 115. 对 OpenAI 姚顺雨 3 小时访谈: 6 年 Agent 研究、人与系统、吞噬的边界、既单极又多元的世界

视频频道: Zhang Xiaojun Podcast

视频时长: 02:31:32

视频链接: <https://www.youtube.com/watch?v=gQgKkUsx5q0>

素材说明: YouTube 无可用字幕; 正文基于 faster-whisper large-v3 CUDA 转写、视频章节、视频描述、封面和自制教学图整理。固定封面视频不在正文重复使用。

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 导读: AI 下半场从模型能力走向系统能力	2
1.1 本章小结	3
2 人: 语言是为了泛化而发明的工具	3
2.1 非共识: 早早押注语言 Agent	4
2.2 本章小结	5
3 系统: 任务、环境与简单通用方法	5
3.1 Agent 的三波兴衰: 方法线和任务线	6
3.2 Code 是关键 affordance	7
3.3 本章小结	9
4 奖励机制: 内生 reward 与 Multi-Agent	9
4.1 Reward 难在哪里	9
4.2 Multi-Agent: 从个体能力到组织结构	10
4.3 本章小结	11
5 Agent 系统设计蓝图: 从研究概念到可运行产品	11
5.1 本章小结	12
6 吞噬边界: Interface、Super App 与 Chatbot 到 Agent	12
6.1 Chatbot 系统为何自然演化成 Agent	13
6.2 Super App 是双刃剑	14
6.3 本章小结	15
7 人类全局: 既单极又多元	15
7.1 Different Bet: 超越霸主需要不同下注	16
7.2 本章小结	17
8 总结与延伸	17
8.1 关键 takeaways	18
8.2 开放问题	18
8.3 拓展阅读	19

1 导读：AI 下半场从模型能力走向系统能力

本节先建立整期的坐标。姚顺雨在 2025 年 4 月发布《The Second Half》，认为 AI 主线程已经进入下半场。上半场的主线更像是模型能力：预训练、规模化、对话和通用能力；下半场则更像系统能力：Agent、任务环境、奖励机制、工具调用、交互界面和组织结构。访谈从个人经历讲起，但真正的核心是一个问题：当模型足够强以后，智能怎样进入世界、形成行动、组织和新的边界？

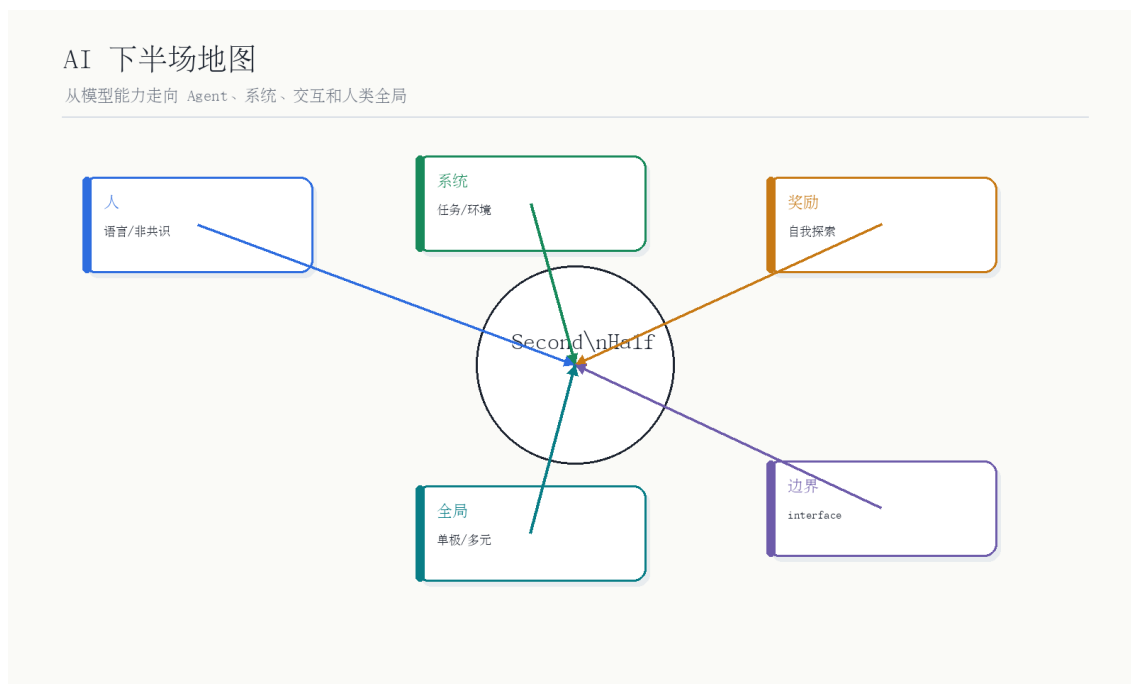


图 1: AI 下半场地图：从模型能力走向 Agent、系统、交互和人类全局。自制概念图，依据 00:01:45–02:31:20 访谈内容整理。

读图：下半场不是“模型不重要”

模型仍然是底座，但讨论重心从单个模型分数转向系统如何行动：任务是否真实、环境是否足够大、reward 是否可定义、interface 是否打开新机会、最终世界是单极还是多元。

本期核心命题

Agent 不是聊天模型的一个功能按钮，而是一种系统范式：它要在环境里观察、推理、行动、获得反馈，并通过工具、奖励和多智能体结构不断扩大可完成任务的边界。

核心概念总表

概念	本期含义	学习时要避免的误解
Agent	与环境交互并试图优化目标的系统	不是“聊天框加几个工具”这么简单。
Environment	Agent 行动和获得反馈的世界	环境太小，能力就无法外推。
Reward	评价动作后果的信号	reward 不等于人类真正想要的全部价值。
Affordance	环境给行动者提供的行动可能性	code/API 是 AI 的手，不只是输出格式。
Interface	用户和智能系统交互的方式	UI 会改变任务、数据、商业和研究方向。
Different Bet	与霸主不同的下注路线	不是换皮复制，而是改变边界条件。

本期阅读路线

路线	核心问题	对应章节
人	为什么语言是泛化工具，为什么姚顺雨押注 Agent？	语言与非共识。
系统	Agent 的任务、环境、方法线如何共同演化？	两个核心问题与三波兴衰。
奖励	Agent 如何拥有 reward、自我探索和 multi-agent 结构？	reward、code affordance、泛化。
边界	Chatbot、interface、Super App 怎样决定新机会？	吞噬边界与交互方式。
全局	单极与多元、OpenAI 与创业机会如何共存？	人类全局与 different bet。

1.1 本章小结

EP115 是 Agent 理论访谈，不只是 OpenAI 研究员访谈。它把语言、人、任务、环境、奖励、交互和平台竞争放在同一张图里，适合作为理解 Agent 下半场的核心笔记。

2 人：语言是为了泛化而发明的工具

本章先看“人”的部分。姚顺雨的一个关键表达是：语言是人为了实现泛化而发明出来的工具，这一点比其他东西更本质。这个观点解释了他为什么从语言出发做 Agent：语言不仅是交流媒介，也是人类压缩经验、传递规则、描述目标、组织计划和跨场景迁移的工具。

语言是泛化工具

人类用语言压缩经验，Agent 用语言连接任务和世界

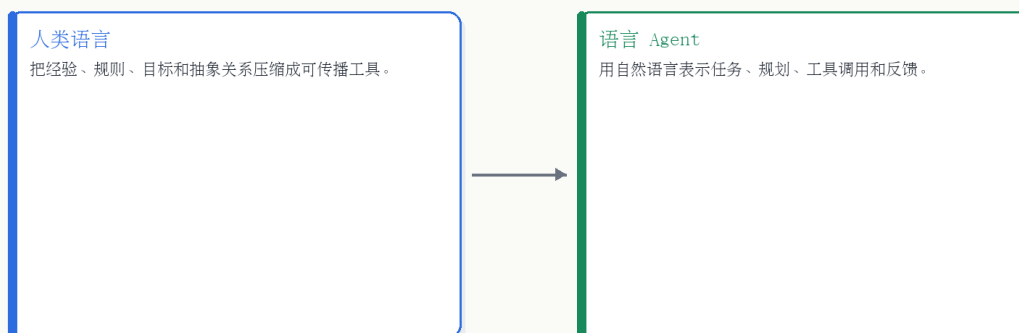


图 2: 语言是泛化工具: 人类用语言压缩经验, Agent 用语言连接任务和世界。自制概念图, 依据 00:02:03-00:07:50 对谈内容整理。

读图: 语言为什么适合做 Agent 的中间层

语言把任务、规则、约束、工具说明和反馈都变成可组合的符号。模型通过语言可以理解人类目标, 也可以调用代码、API、文档和其他 Agent。因此语言不是 Agent 的全部, 但它是很强的通用接口。

2.1 非共识: 早早押注语言 Agent

上一节解释语言为什么是泛化工具, 本节看这个判断如何变成研究下注。姚顺雨说自己一直有一个非共识: 想要做 Agent, 而且第一件事就是基于语言模型做 Agent。在当时, 主流路线并不一定认为语言模型是通向 Agent 的最佳路径; 但后来 GPT 系列显示, 语言模型拥有强泛化和工具接口潜力, 这个非共识变成了前沿主线。

Different Bet 的意义

如果只复制已有霸主的主线, 很难超越霸主。新的机会往往来自 different bet: 不同任务、不同环境、不同 interface、不同产品形态。语言 Agent 在早期就是这样一种下注。

为什么早期语言 Agent 是非共识

早期语言模型看起来更像文本系统, 而不是能行动的智能体。要把它看成 Agent, 需要同时相信三件事: 语言能表达任务, 模型能跨任务泛化, 外部工具可以把语言计划变成动作。这三件事在当时都不是显然共识。

阶段	关注问题	与 Agent 的关系
博士早期	用语言模型做简单游戏和任务	证明语言可以承载决策和状态。
任务环境	寻找更真实、更有价值的 benchmark	让 Agent 不只是刷小题。
工具/代码	InterCode、代码环境、数字世界 affordance	让 Agent 获得可执行的手。
下半场	reward、multi-agent、interface、系统演化	从单模型能力进入系统能力。

2.2 本章小结

“人”的章节说明，Agent 研究不是凭空从模型能力里长出来的，而是来自对语言、人类泛化和任务表达的理解。语言之所以重要，是因为它把人类目标和机器行动接在一起。

3 系统：任务、环境与简单通用方法

上一章讲为什么语言是入口，本章进入 Agent 作为系统的核心。姚顺雨把自己的研究概括成两个问题：第一，怎样做有价值、和现实世界更相关的任务和环境；第二，怎样做简单但通用的方法。前者是任务线，后者是方法线。很多讨论只看方法线，忽略任务线，但 Agent 的能力往往被环境定义。

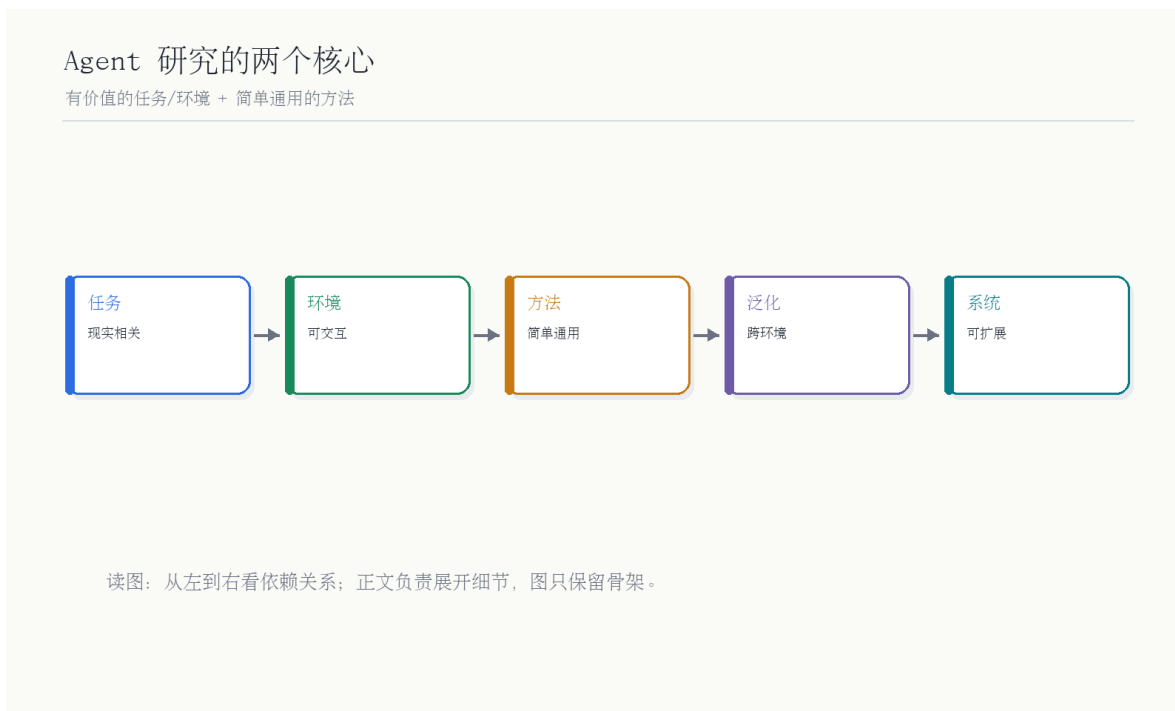


图 3: Agent 研究的两个核心：有价值的任务/环境 + 简单通用的方法。自制概念图，依据 00:17:50–00:35:00 对谈内容整理。

读图：任务和方法必须共同进化

如果任务太简单，方法看起来很强但没有现实意义；如果任务足够真实但方法太复杂，就很难泛化。Agent 研究要同时设计环境、奖励、工具和通用方法。

3.1 Agent 的三波兴衰：方法线和任务线

上一节说任务和方法要共同进化，本节把这个判断放回 Agent 历史。Agent 是一个古老概念：能自我决策、与环境交互并优化奖励的系统，都可以被称为 Agent。访谈提醒我们，Agent 的历史不是一条直线，而是多次兴衰：古典 AI/RL Agent、强化学习环境、再到 LLM Agent。每一波都不只是方法变化，也依赖任务和环境是否足够合适。



图 4: Agent 三波兴衰: 方法线和任务线相辅相成, 不能只看算法。自制概念图, 依据 00:17:50–00:29:00 对谈内容整理。

术语消化: Agent 系统的基本构件

构件	含义	为什么重要
Environment	Agent 观察和行动的世界	决定任务是否真实、反馈是否有效。
Policy	根据观察选择动作的策略	可以由 LLM、代码、规划器或多模型系统实现。
Reward	对动作结果的评价信号	决定 Agent 学什么、探索什么。
Memory	长期状态和经验记录	支撑跨任务、跨会话和个性化。
Tools	外部 API、代码、浏览器、文件等能力	让 Agent 从语言进入可执行世界。

方法线 vs 任务线

线索	关注什么	容易忽略什么
方法线	算法、模型、规划、训练技巧	任务是否足够真实, 环境是否能反馈。
任务线	benchmark、环境、工具、数据和评价	方法是否简单通用, 能否迁移到新任务。
系统线	方法和任务如何形成闭环	计算成本、失败恢复、长期记忆和安全。

这张表也解释了为什么许多 Agent demo 看起来惊艳, 却难以成为稳定产品。Demo 往往只展示方法线: 模型会规划、会调用工具、会写几步动作; 真实产品还要处理任务线: 环境是否可重复、反馈是否可判定、失败是否可恢复、用户是否愿意把真实目标交给它。姚顺雨强调任务和环境, 正是在提醒 Agent 研究不能只追求“会做样子”。

Agent benchmark 的设计标准

一个好的 Agent benchmark 至少要满足三点: 任务足够真实, 反馈足够明确, 环境足够开放。太简单的任务测不出泛化; 太封闭的环境训练不出真实行动; 没有可靠反馈的任务又无法形成学习信号。

3.2 Code 是关键 affordance

前面讨论 Agent 的构件, 本节进入“行动接口”。访谈里有一个非常重要的判断: code 像人的手, 是 AI 最重要的 affordance。Affordance 指环境给予行动者的行动可能性。对 AI 来说, 代码、API、命令行、浏览器和文件系统让模型不只是说话, 而是能够改变数字世界。

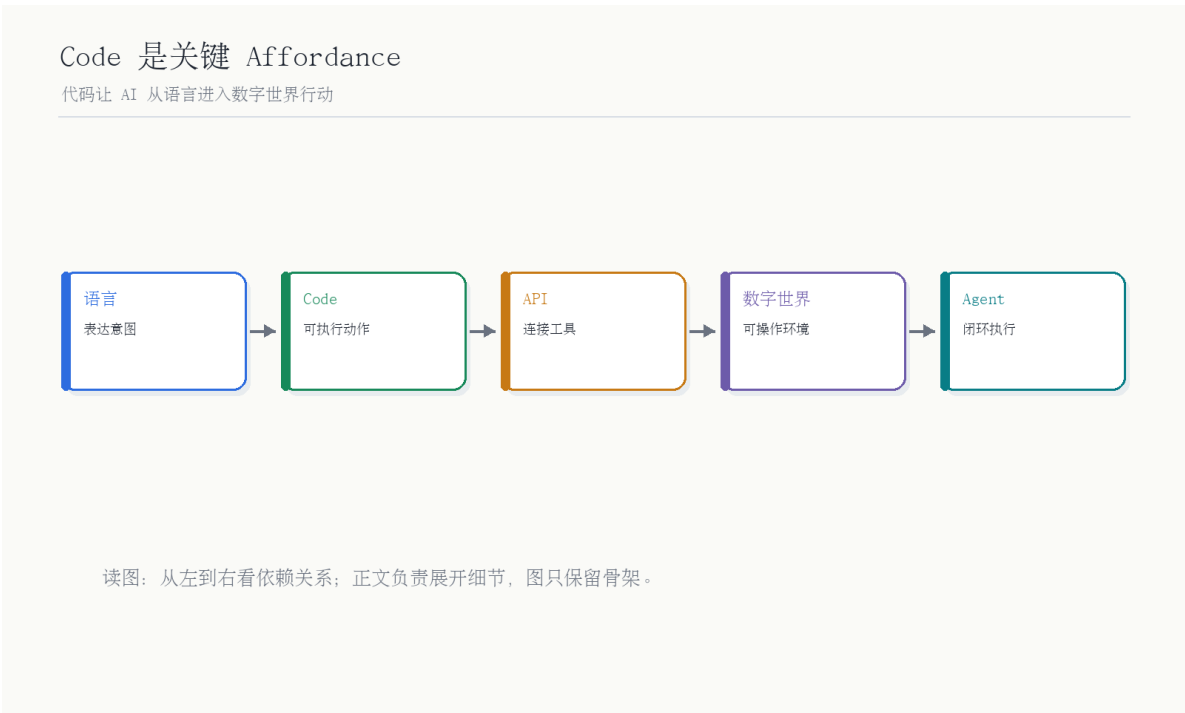


图 5: Code 是关键 Affordance: 代码让 AI 从语言进入数字世界行动。自制概念图，依据 00:29:49–00:33:00 对谈内容整理。

为什么 code 是手

语言表达意图，code 执行动作。只会语言的模型像“会想会说的人”；会写代码、调用工具和修改环境的 Agent，才开始拥有数字世界里的“手”。

术语消化: Affordance

Affordance 原本来自生态心理学，指环境向行动者提供的行动可能性。门把手给人“拉开门”的 affordance；按钮给人“点击”的 affordance；对 AI 来说，代码、API、命令行和浏览器提供“改变数字世界”的 affordance。

数字世界 affordance 的层级

层级	例子	Agent 能做什么
文本层	文档、网页、邮件	读取、摘要、改写、检索。
代码层	Python、SQL、shell、API	计算、查询、调用服务、修改系统状态。
界面层	浏览器、IDE、操作系统	跨工具执行任务，处理非结构化环境。
组织层	多人协作、权限、审批	与人类流程共同完成高风险任务。

3.3 本章小结

Agent 系统的关键是任务、环境、方法和 affordance 的共同设计。语言模型提供认知能力，代码和工具提供行动能力，环境与 reward 提供学习方向。

4 奖励机制：内生 reward 与 Multi-Agent

本章进入 reward。姚顺雨提到，Agent 发展的关键方向之一，是让它拥有自己的 reward，能自己探索；另一个方向是 Multi-Agent，让它们之间形成组织结构。这里的 reward 不只是 RL 公式，而是 Agent 能否从环境中获得有效反馈、能否不只模仿人类数据、能否在任务中自我改进的核心。

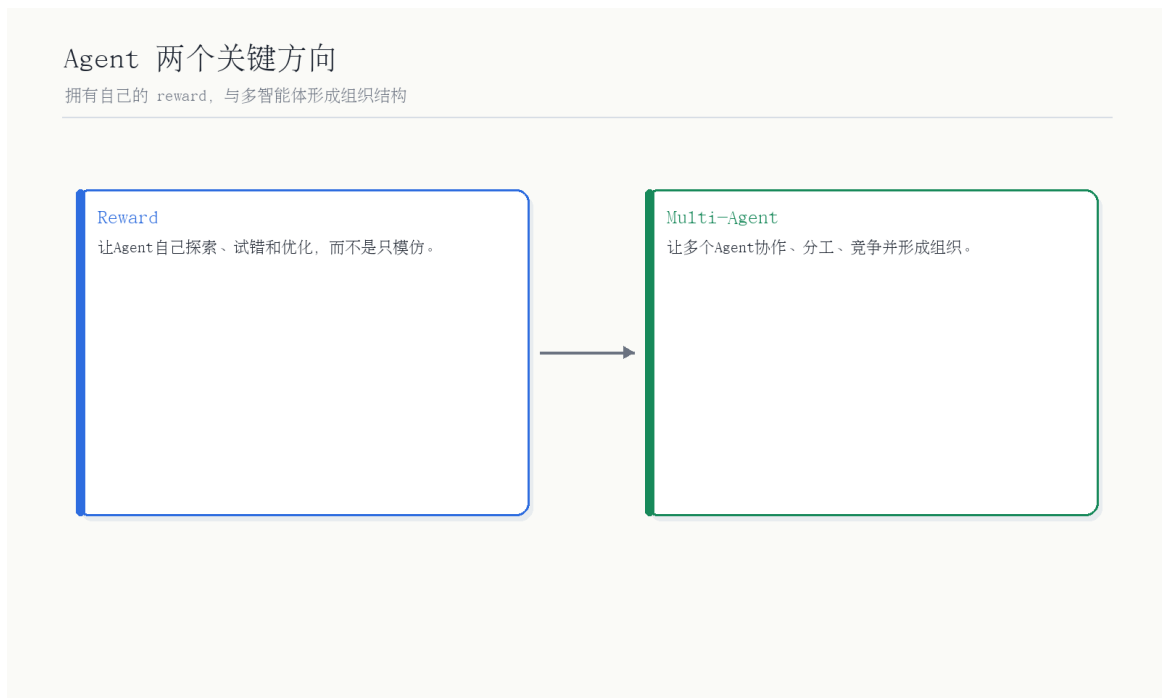


图 6: Agent 两个关键方向:拥有自己的 reward,与多智能体形成组织结构。自制概念图,依据 00:27:19–01:03:00 对谈内容整理。

读图: reward 和 multi-agent 是两条不同扩展路径

Reward 让单个 Agent 能探索、试错和优化; Multi-Agent 让多个 Agent 通过分工、协作和竞争形成更复杂的组织。前者解决学习信号, 后者解决组织结构。

4.1 Reward 难在哪里

上一节把 reward 和 multi-agent 分成两条扩展路径, 本节先看 reward 为什么难。现实任务的 reward 很难定义。棋类游戏有明确胜负, 代码任务有测试, 网页任务可能有完成状态; 但很多现实任务的目标长期、稀疏、主观或不可完全观测。reward 设计不好, Agent 可能优化错目标, 或者钻评价漏洞。

奖励定义问题

现实任务的 reward 难在长期、稀疏和不可完全观测



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 7: 奖励定义问题:现实任务的 reward 难在长期、稀疏和不可完全观测。自制概念图,依据 00:40:00–01:03:00 对谈内容整理。

Reward hacking 的风险

如果 reward 只奖励表面结果, Agent 可能学会投机。例如只奖励点击率, 它可能制造标题党; 只奖励任务完成, 它可能绕过权限; 只奖励测试通过, 它可能写脆弱代码。企业和真实世界任务尤其需要审计和安全约束。

Reward 的直觉公式

$$\text{learning signal} = r(s_t, a_t, s_{t+1})$$

其中, s_t 是行动前状态, a_t 是 Agent 的动作, s_{t+1} 是行动后状态, r 是评价这个变化的奖励函数。现实任务难在: 状态不完整、动作后果延迟、评价标准常常由人和组织共同决定。

4.2 Multi-Agent: 从个体能力到组织结构

Multi-Agent 的关键不是“多开几个模型实例”, 而是让 Agent 之间形成分工、通信、协作、竞争和组织结构。它接近人类组织, 也会带来新的问题: 谁分配任务, 谁验证结果, 谁承担冲突, 如何避免群体幻觉。

术语消化: Reward 与 Multi-Agent

概念	解决的问题	常见风险
External reward	环境或人类给出的外部评价	昂贵、稀疏、容易被过拟合。
Intrinsic reward	Agent 自己形成探索信号	可能偏离人类目标。
Multi-Agent	多个 Agent 分工协作	通信成本、冲突和责任边界。
Verification	验证 Agent 输出是否可靠	验证器本身也可能出错。

Reward 和 multi-agent 的难点还在于它们会改变系统的“社会性”。单个 Agent 犯错时，问题可以追溯到单个策略；多个 Agent 协作时，错误可能来自任务分配、通信误解、局部最优、验证器缺陷或激励不一致。越接近真实组织，Agent 系统越需要治理层，而不是只需要更强模型。

多智能体不是自动等于更强

多 Agent 系统可能带来分工和并行，也可能带来更高通信成本、更复杂失败模式和责任不清。只有当任务天然可分解、验证机制清楚、协作协议稳定时，多智能体才更可能超过单 Agent。

4.3 本章小结

Reward 决定 Agent 学什么，Multi-Agent 决定 Agent 如何组织。下半场的难点不是让模型更会聊天，而是让系统能在真实环境里安全地探索和协作。

5 Agent 系统设计蓝图：从研究概念到可运行产品

前面几章分别讨论语言、任务环境、reward 和 multi-agent，本章把它们合成一个可设计的系统蓝图。一个 Agent 产品通常不是单个 prompt，而是一套循环：用户目标进入系统，系统把目标转成任务状态，模型生成计划，工具执行动作，环境返回观察，验证器判断结果，记忆层保存经验，最后再进入下一轮。

Agent 系统最小闭环

Goal → State → Plan → Action → Observation → Reward/Verification → Memory

其中，Goal 是用户目标，State 是当前任务状态，Plan 是模型生成的计划，Action 是工具或代码执行，Observation 是环境反馈，Reward/Verification 是评价信号，Memory 是长期经验和上下文。

设计蓝图：每一层问什么

层级	设计问题	常见失败
目标层	用户到底要什么？成功条件是什么？	目标含糊，Agent 做了看似有用但无关的事。
状态层	当前有哪些文件、工具、权限和约束？	上下文缺失，动作建立在错误状态上。
计划层	是否需要拆任务、并行或询问用户？	计划太长、太脆弱，遇到异常不会改。
行动层	哪些动作能安全执行，哪些必须确认？	越权、误删、调用错误工具。
验证层	如何知道任务真的完成了？	只看表面输出，不验证真实结果。
记忆层	哪些经验应该保留到下次？	记忆污染、隐私风险、过期信息。

从 demo 到产品的距离

Agent demo 可以靠一次顺利轨迹打动人；Agent 产品必须处理失败恢复、权限、日志、验证、用户中断、长期记忆和成本。下半场的系统能力，很大一部分就在这些“无聊但必要”的层里。

5.1 本章小结

Agent 系统设计的核心不是把模型包一层 UI，而是把目标、状态、计划、行动、观察、验证和记忆闭合起来。只有闭环稳定，Agent 才能从惊艳演示变成可靠产品。

6 吞噬边界：Interface、Super App 与 Chatbot 到 Agent

上一章讲 Agent 内部机制，本章讲外部边界。姚顺雨认为创业公司的最大机会，是设计不同 interface。模型能力可能产生 beyond ChatGPT 的交互方式，变成新的 Super App。这里的“吞噬边界”不是说一个公司会吃掉所有东西，而是说不同交互界面会决定智能系统能进入哪些任务。



图 8: 吞噬边界：Interface 决定机会，不同交互方式可能产生不同 Super App。自制概念图，依据 00:48:38–01:05:01 对话内容整理。

读图：interface 是边界，不只是 UI

Chatbot、Assistant、Her、非人形界面、浏览器、IDE、操作系统都可能成为智能入口。不同 interface 会引导不同任务、数据和商业模式，也会塑造研究方向。

6.1 Chatbot 系统为何自然演化成 Agent

模型公司的 Chatbot 系统天然有用户意图、上下文、工具调用需求和长期状态需求。因此它会自然演化成 Agent 系统：先记住用户，再调用工具，再执行任务，再把反馈写回系统。Chatbot 不是终点，而是 Agent 的入口形态之一。



图 9: Chatbot 到 Agent: 模型公司的聊天系统会自然演化成 Agent 系统。自制概念图, 依据 01:49:28–02:05:00 对谈内容整理。

Chatbot 到 Agent 的最小迁移

一个 Chatbot 只要加入长期 memory、工具调用、任务状态和结果反馈, 就会开始变成 Agent。区别不在界面是否还是聊天框, 而在系统是否能持续改变外部状态。

6.2 Super App 是双刃剑

上一节讲 Chatbot 如何自然演化成 Agent, 本节看强入口带来的组织路径依赖。拥有 ChatGPT 这样的 Super App 是巨大优势, 因为它带来用户、数据、分发和产品反馈; 但它也会塑造研究方向, 让组织自然围绕这个 Super App 优化。创业公司如果只是复制旧 interface, 很难超越; 如果找到不同 interface, 就可能打开新任务边界。

界面锁定会影响研究路线

当公司拥有强入口, 它的研究很容易服务这个入口。这不是坏事, 但会让它低估其他 interface 的机会。历史上的平台替代, 常常不是旧平台的更好版本, 而是完全不同的交互方式。

Interface 类型表

Interface	典型任务	潜在机会
Chatbot	问答、写作、解释、轻量任务	通用入口强，但容易同质化。
IDE/Code	编程、调试、自动化工具	affordance 强，任务可验证。
Browser/OS	跨网页、文件和应用的数字行动	可能形成 universal digital agent。
Non-human UI	非人形、嵌入式、背景式交互	可能避开 Chatbot 路径锁定。

因此，“吞噬的边界”可以理解为 interface 的边界。一个模型如果只能在聊天框里工作，它吞噬的是问答、写作和轻量任务；如果它进入 IDE，它吞噬的是代码生产；如果它进入浏览器和操作系统，它吞噬的是跨应用数字行动；如果它进入企业流程，它吞噬的是组织中的重复协调。边界扩张不是模型自己完成的，而是模型、界面、工具和数据共同完成的。

6.3 本章小结

智能边界不是由单一模型决定，而是由 interface、任务、工具、用户习惯和组织路径共同决定。Agent 的机会，也会从新的交互方式里长出来。

7 人类全局：既单极又多元

上一章讨论 interface 如何决定智能边界，本章收束到平台竞争和人类全局。姚顺雨认为 OpenAI 可能成为类似 Google 的重要公司，但这不代表世界会被单极系统垄断。模型、算力、数据和 Super App 会集中，这是单极趋势；但不同 interface、不同任务、不同组织结构和 different bet 又会制造多元机会。

既单极又多元

重要模型公司会成为关键一环，但世界不必被单极垄断

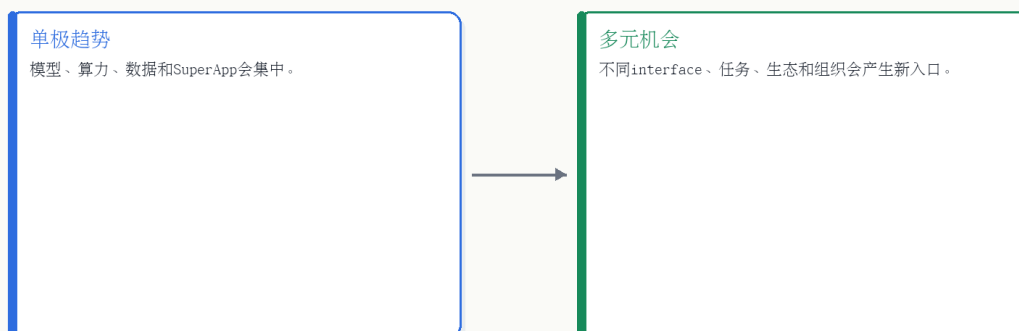


图 10: 既单极又多元: 重要模型公司会成为关键一环, 但世界不必被单极垄断。自制概念图, 依据 01:35:00-02:31:20 对谈内容整理。

读图: 单极和多元可以同时成立

基础模型和超级入口可能高度集中; 但智能系统进入世界的方式很多, 任务、界面、工具、行业和组织差异会不断制造新入口。因此“有强中心”和“有多元生态”并不矛盾。

500 亿美金 allocate 问题

下注方向	可能买到什么	风险
Frontier model	模型能力和算力规模	与现有巨头正面竞争, 资本密度极高。
Super App	用户入口和数据回流	容易被已有强入口牵引, 路径依赖强。
Different interface	新任务、新交互、新生态	不确定性高, 但上限可能更高。
Public-good re-search	安全、评测、开放环境和工具	商业回报慢, 但人类贡献更直接。

7.1 Different Bet: 超越霸主需要不同下注

前面说世界既有集中趋势也有多元机会, 本节解释多元机会从哪里来。访谈开头和结尾都反复出现 different bet。姚顺雨举例说, 如果 OpenAI 一直做强化学习, 也很难超过 DeepMind; 要超越霸主, 通常要下注到不同路径上。今天的创业机会也类似: 如果只复制 ChatGPT 或 Claude, 很难; 如果设计不同 Super App、不同 interface、不同任务环境, 就仍有空间。

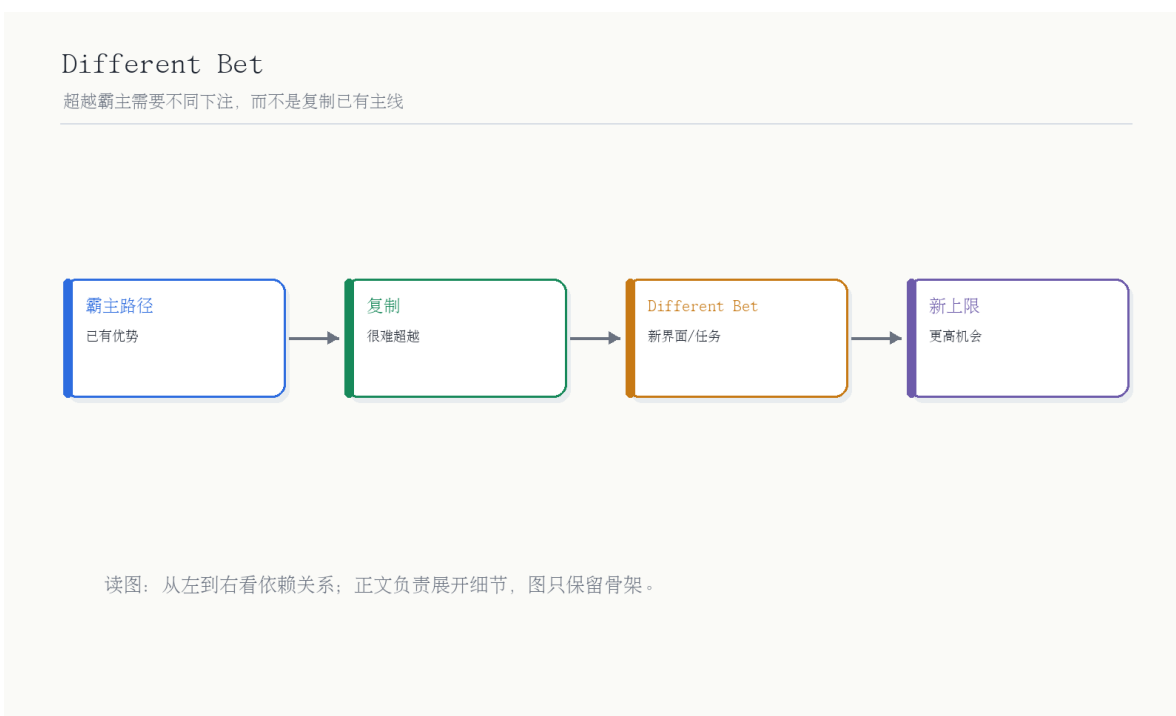


图 11: Different Bet: 超越霸主需要不同下注，而不是复制已有主线。自制概念图，依据 00:00:11–00:01:23 与 02:30:20–02:30:37 对谈内容整理。

创业机会的判断

真正的危险不是“一个类似微信的东西打败微信”，而是一个不一样的东西打败微信。AI 时代的新机会也更可能来自新任务、新界面和新组织，而不是复制已有聊天产品。

Different Bet 的判断清单

一个下注是否真的 different，可以问四个问题：它是否改变了用户入口？是否改变了任务环境？是否改变了反馈和 reward？是否改变了组织结构或商业模式？如果只是把同样聊天框换个皮肤，它通常不是 different bet。

7.2 本章小结

“既单极又多元”是这期最重要的平台判断。强模型公司会成为关键节点，但智能系统最终进入世界的边界，仍由不同 interface 和 different bet 决定。

8 总结与延伸

本节把 EP115 压缩成四个结论。第一，语言是泛化工具，所以语言模型天然适合作为 Agent 的任务表示和工具接口。第二，Agent 研究不能只看方法，还要看任务和环境；没有好环境，方法无法证明现实价值。第三，reward 和 multi-agent 是下半场的重要方向，但也带来安全、验证和组织问题。第四，interface 决定智能进入世界的边界，Super App 既是优势也是路径锁定。

把 EP115 放进张小珺 AI 队列

EP139 讲 Agent 技术史, EP138 讲 Agent 后训练和组织平权, EP116 讲企业级 Agentic Model, EP115 则给出更底层的研究框架: 任务/环境、简单通用方法、reward、multi-agent、code affordance 和 interface。

与前后几集的关系

节目	关注点	与 EP115 的连接
EP139	Agent 技术史和 OpenClaw moment	给出 Agent 的历史谱系, EP115 给出研究框架。
EP138	Agent 后训练、环境和 rollout	展开下半场为什么需要环境和反馈。
EP116	企业级 Agentic Model 和可信执行	把 EP115 的 Agent 系统放进 ToB 工作流。
EP118	Agent OS、VLA 与平台终端	把 interface 和 Agent 系统放进物理世界与操作系统。

8.1 关键 takeaways

1. Agent 是能与环境交互并优化目标的系统, 不是简单聊天功能。
2. 任务和环境与方法同等重要, 甚至常常决定研究是否有现实意义。
3. Code 是 AI 在数字世界里的关键 affordance, 是语言到行动的桥梁。
4. Reward 难定义, Multi-Agent 难组织, 二者都是下半场核心难题。
5. 新机会来自 different bet: 不同界面、不同任务、不同组织, 而不是复制霸主。

8.2 开放问题

前面的 takeaways 是确定性较高的结论, 本节保留几个仍然开放的问题。它们之所以重要, 是因为 Agent 下半场不是一条已经铺好的路; reward、multi-agent、interface 和平台结构都还在快速变化, 任何一个问题的答案改变, 都可能改变下一代产品和研究路线。

1. Agent 的内生 reward 怎样设计, 才能鼓励探索而不偏离人类目标?
2. Multi-Agent 系统中的责任如何划分, 谁来验证最终结果?
3. Chatbot、IDE、Browser、OS、机器人等 interface 会不会形成不同类型的 Super App?
4. 当模型公司拥有强入口时, 它们如何避免被现有 interface 锁定?
5. 如果世界既单极又多元, 创业公司应该在哪些任务和交互方式上下注?

开放问题不是结尾装饰

这些问题会直接影响研究和产品: reward 决定训练方向, multi-agent 决定组织形式, interface 决定入口和数据, 平台结构决定创业机会。把问题保留下来, 比提前给一个漂亮但脆弱的结论更诚实。

8.3 拓展阅读

- 继续对照 EP139 Agent 综述, 可把姚顺雨的研究框架放进更长的 Agent 技术史。
- 继续对照 EP138 罗福莉访谈, 可理解 Agent 阶段为什么后训练、环境和 rollout 变得更重要。
- 继续对照 EP116 吴明辉访谈, 可观察 Agentic Model 在企业服务中的私有数据和可信执行版本。